



CEF488: Systems and Network Programming

LAB5: Distributed Computing (Cooperative Image Rendering)

GROUP 8 MEMBERS

ECHUNDIP CARL-LAWRENCE	FE23A040
NSEMEN PHILLIS NANA	FE25A679
OBIOMA OBI FODJO	FE23A133
BERINYUY GILLES KILALON	FE23A024
TAMBE RAMSON	FE23A156

Lecturer: Engr Forcha Glen

2025-2026 academic year/ Semester 2

June,2026

Table of Contents

1. Honesty report	3
2. Abstract:.....	3
3. Objectives:	3
4. Pre-lab Questions:	4
5. Methodology:	6
6. Implementation:	7
a. Task_1: Creating the library, Mandelbrot.....	7
b. Task_2: Master Program:	11
c. Task_3: Worker Program:	12
d. Task_4: Program Implementation:	13
✓ Test of Task4:	14
e. Task_5: Load Balancing:.....	15
f. Task_6: Performance Analysis:	17
7. Conclusion	18

1.Honesty report

Task	Responsible member	Contributions	Reviewers
Prelab Questions	TAMBE RAMSON	NSEMEN PHILLIS NANA	OBIOMA OBI FODJO
Implementations	ECHUNDIP CARL- LAWRENCE	NSEMEN PHILLIS NANA	BERINYUY GILLES KILALON
Testing and debugging	OBIOMA OBI FODJO NSEMEN PHILLIS NANA	BERINYUY GILLES KILALON	TAMBE RAMSON
Report Writing	BERINYUY GILLES KILALON ECHUNDIP CARL- LAWRENCE	NSEMEN PHILLIS NANA	TAMBE RAMSON OBIOMA OBI FODJO

2.Abstract:

With this lab, we were charged with the design and implementation of a distributed computing system which can jointly renders a set of fractal image across multiple machines.

We used a master to client architecture is used where one master-process process distributions row-level works until when 3 worker processes over TCP sockets using a custom binary protocol.

Two work-distribution strategies are explored: static assignment (fixed row ranges per worker) and dynamic load balancing (a shared work queue of chunks). Fault-tolerance mechanisms using epoll timeouts detect unresponsive workers and trigger task reassignment.

Performance measurements here enable us to demonstrate near-linear speedup for up to three workers, with overhead from network transfer and synchronization forming the primary bottleneck. The implementation is written in C across five source modules and produces a pixel-verified PPM image identical to single-machine output.

3.Objectives:

Our main objectives for this lab goes as thus:

- Designing and implementing a master to worker distributed system communicating over TCP sockets.

- Encoding and decoding a custom binary protocol with correct network byte ordering for integers and IEEE 754 doubles.
- Parallelize an embarrassingly parallel computation (Mandelbrot rendering) across multiple networked machines.
- Apply both static work partitioning and dynamic work-queue scheduling and compare their performance.
- Also, implementing fault tolerance using socket timeouts and epoll-based I/O monitoring to detect and recover from worker failures.
- Measure and analyse distributed system performance using speedup and efficiency metrics.
- Produce a correct, pixel-verified output image assembled from distributed partial results.
- Write robust, well-commented C code that handles socket errors, partial sends/receives, and connection failures gracefully.

4.Pre-lab Questions:

1. What is the Mandelbrot Set? How Do You Determine Whether a Point Belongs to It?

The Mandelbrot set is the set of all complex numbers where the iterative sequence defined by $z_0 = 0$ and $z_{n+1} = z_n^2 + c$ remains bounded — that is, $|z_n|$ does not diverge to infinity as n increases.

To determine membership of a point $c = c_{re} + c_{im} \cdot i$:

- I. Set $z = 0 + 0i$.
- II. Compute $z = z^2 + c$
- III. If $|z|^2 = z_{re}^2 + z_{im}^2 > 4$, the point has escaped; then record iteration count n .
- IV. If n reaches N_{max} without escape, the point is considered inside the set.

2. Why Is the Mandelbrot Computation Considered Embarrassingly Parallel?

A problem is said to be embarrassingly parallel when it decomposes into independent subtasks that share no data and require no communication between themselves during execution. The Mandelbrot computation satisfies this exactly: the escape iteration count for pixel (row r , col c) depends only on its corresponding complex coordinate, which is computed deterministically from (r, c) and the global rendering parameters.

3. Describe the Master Worker Architecture and the Responsibilities of Each Component

Master responsibilities:

The following are the responsibilities of the Master

- Parses global rendering parameters (image dimensions, Mandelbrot parameters, port number).
- Creates a TCP listening socket and waits for worker connections.
- Maintains a global pixel buffer and a work queue of row chunks.

Worker responsibilities:

- Connects to the master using the provided IP address and port.
- Receives a task packet specifying image dimensions, row range, and Mandelbrot parameters.
- For each assigned row, calls `compute_row()` to fill a local pixel buffer.

4. Why Is It Important to Include a Length Field in Each Binary Protocol Message?

TCP is a stream-oriented protocol provides no built-in notion of message boundaries. A single `send()` call may be split across multiple segments by the network stack, and multiple `send()` calls may be coalesced by Nagle's algorithm. Without a length field, the receiver cannot determine where one message ends and the next begins. Including a fixed-size header containing the payload length allows the receiver to first read the header (a known number of bytes), extract the length, and then loop on `recv()` until exactly that many payload bytes are accumulated.

5. How Would You Handle Partial Reads/Writes When Sending Large Amounts of Pixel Data?

Both `send()` and `recv()` may transfer fewer bytes than requested, especially for large payloads such as pixel arrays. The correct pattern is to wrap them in a loop that tracks total bytes transferred and retries until the full count is reached.

6. Trade-Offs Between Static Work Distribution and Dynamic Work Queues

Aspect	Static Assignment	Dynamic Queue
Master overhead	Minimal (assign once at start)	.Continuous (respond to each request)
Load balance	Poor if workers differ in speed	Self-balancing; faster workers get more
Fault recovery	Reassign the whole block	Return chunk to queue

7. How Does a Timeout Help in Fault Tolerance? What Could Go Wrong?

When the master sends a task to a worker and waits indefinitely for a response, a crashed or network-disconnected worker will stall the entire computation forever. A timeout implemented via `epoll_wait()` with a millisecond deadline causes the master's wait to expire after a fixed period. If no data arrives within that window, the master assumes the worker has failed, closes its socket, and places the assigned rows back in the work queue for redistribution.

8. What Is Speedup? What Factors Limit It in a Distributed Computation?

Speedup $S(p)$ is defined as $S(p) = T_1 / T_p$, where T_1 is the wall-clock time for the best sequential algorithm and T_p is the wall-clock time using p workers. Perfect (linear) speedup means $S(p) = p$. Efficiency $E(p) = S(p) / p$ measures how effectively each worker is used (ideal: 1.0).

- Load imbalance: With static assignment, if one worker's rows contain denser Mandelbrot regions (more iterations per pixel), it finishes later than others, causing idle time.
- Synchronization: The master cannot write the final file until all rows are received; the slowest worker determines overall latency.
- Network bandwidth saturation: Transferring large pixel arrays from multiple workers simultaneously may exceed NIC or switch capacity.
- Context switching and OS scheduling on shared machines.

5.Methodology:

This laboratory helped us to use an incremental development approach to build a distributed Mandelbrot fractal rendering system in C, progressing through four phases.

First, the Mandelbrot computation was implemented and validated in isolation as a self-contained library that computed pixel rows and saved PPM image files. A sequential test program rendered the full 800×600 image locally to produce a reference output used as a correctness benchmark throughout the lab.

Second, a custom binary TCP protocol was designed with four message types: task assignment, result submission, work request, and termination each prefixed with an 8-byte header carrying an opcode and payload length. Reliable send and receive wrappers handled TCP's partial transfer behaviour, and all numeric fields were properly byte-order converted for cross-platform correctness.

Third, the worker and master programs were implemented. The worker connected to the master, sent explicit work requests, computed assigned row chunks using the Mandelbrot library, and returned results in a loop until receiving termination. The master used Linux `epoll` to monitor all worker sockets simultaneously, maintained a dynamic work queue of 10-row

chunks, dispatched chunks on demand, assembled results into a global image buffer, and detected failed workers through epoll timeouts, returning their unfinished chunks to the queue for reassignment.

Finally, performance was measured by timing the sequential baseline against distributed runs with one, two, and three workers. Speedup and efficiency were calculated for each configuration and analysed against Amdahl's Law, with bottlenecks identified as network transfer overhead, the sequential image assembly step, computational load imbalance across image regions, and synchronization delay from the slowest worker.

6.Implementation:

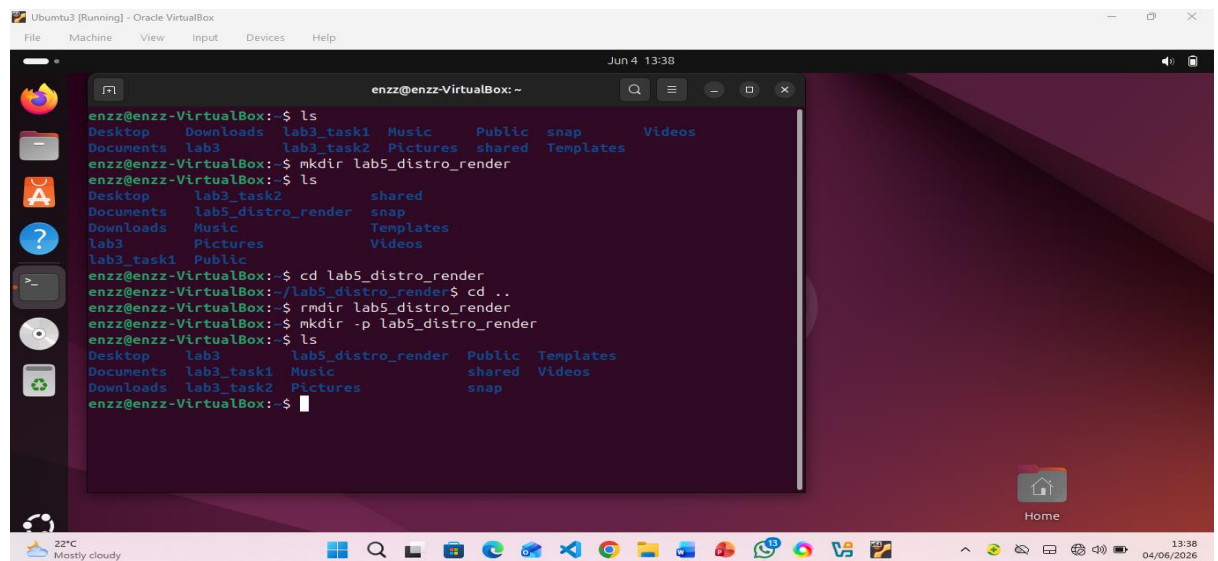
a. Task_1: Creating the library, Mandelbrot

This library is responsible for providing the function which computes one row of pixels stores the iteration count in a pixel array. With coordinates:

$$X = \text{centerx} + (\text{col-width}/2) * \text{zoom}$$

$$Y = \text{centery} - (\text{row-height}/2) * \text{zoom}$$

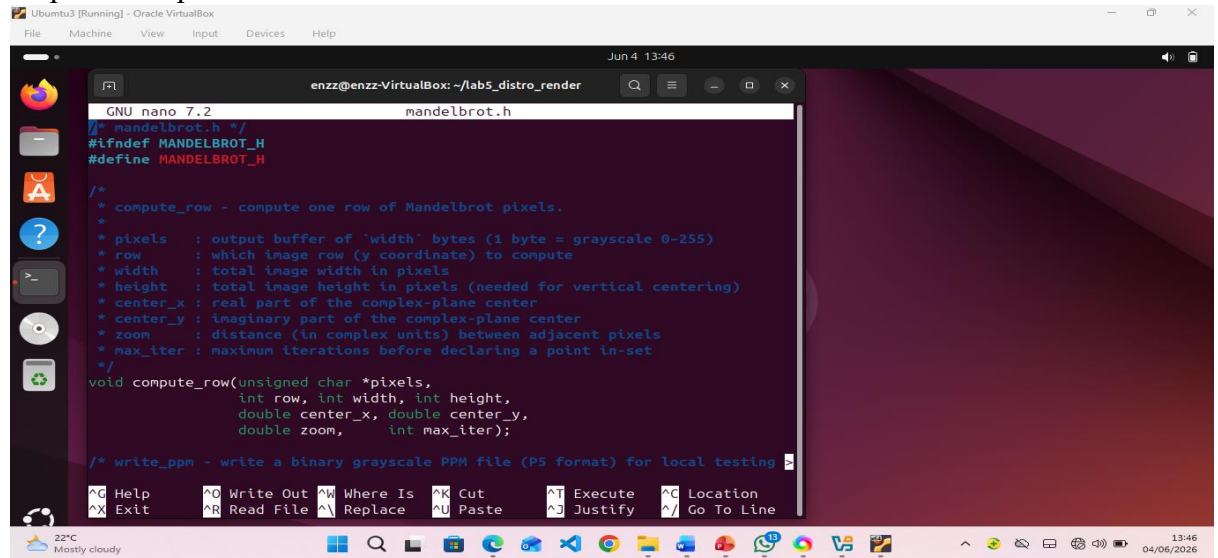
The first thing we did into this task was creating our project directory for the entire lab5 named it “lab5_distro_render”



```
enzz@enzz-VirtualBox: ~  
$ ls  
Desktop  Downloads  lab3_task1  Music      Public  snap  Videos  
Documents lab3        lab3_task2  Pictures   shared  Templates  
enzz@enzz-VirtualBox: ~$ mkdir lab5_distro_render  
enzz@enzz-VirtualBox: ~$ ls  
Desktop  lab3_task2  shared  
Documents lab5_distro_render  snap  
Downloads Music        Templates  
lab3      Pictures    Videos  
lab3_task1 Public  
enzz@enzz-VirtualBox: ~$ cd lab5_distro_render  
enzz@enzz-VirtualBox: ~/lab5_distro_render$ cd ..  
enzz@enzz-VirtualBox: ~$ rmdir lab5_distro_render  
enzz@enzz-VirtualBox: ~$ mkdir -p lab5_distro_render  
enzz@enzz-VirtualBox: ~$ ls  
Desktop  lab3      lab5_distro_render  Public  Templates  
Documents lab3_task1 Music        shared  Videos  
Downloads lab3_task2 Pictures    snap
```

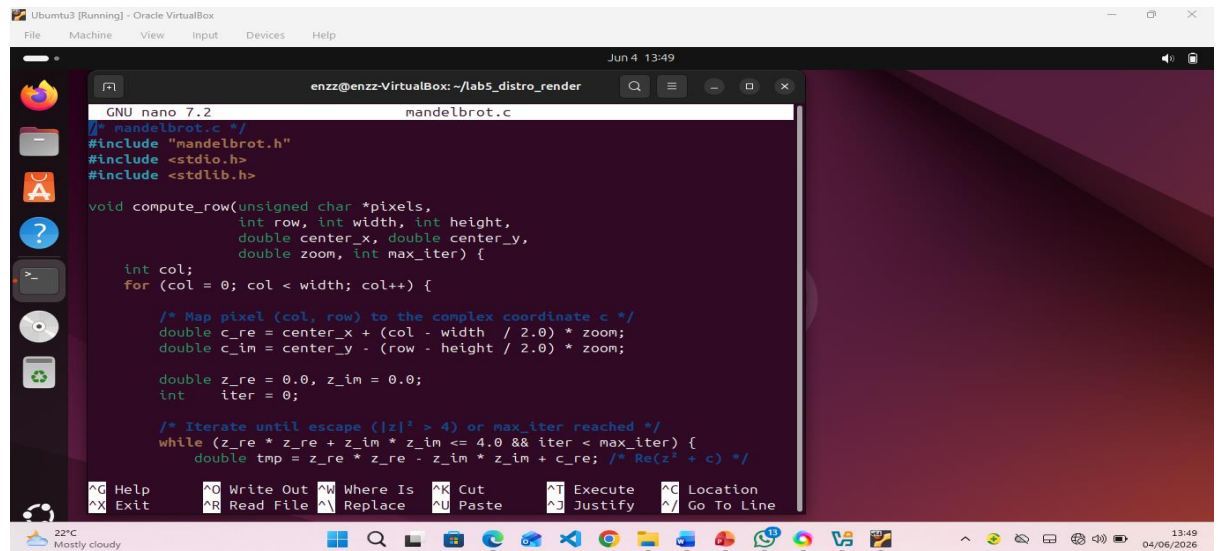
After creating our folder, we moved on to creating a header file and making sure it has all the settings which are necessary for our system since its main goal is to handle

all pixel computations. We named it Mandelbrot.h:

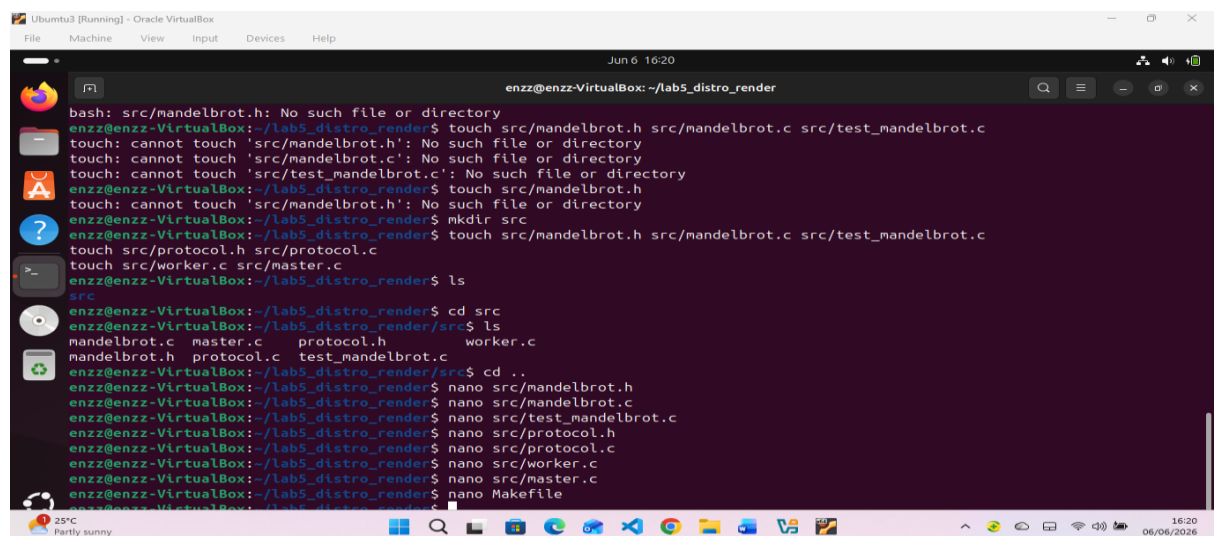


```
GNU nano 7.2 mandelbrot.h
/*
 * mandelbrot.h
 *
 * #ifndef MANDELBROT_H
 * #define MANDELBROT_H
 *
 * /*
 *  * compute_row - compute one row of Mandelbrot pixels.
 *  *
 *  * pixels : output buffer of 'width' bytes (1 byte = grayscale 0-255)
 *  * row : which image row (y coordinate) to compute
 *  * width : total image width in pixels
 *  * height : total image height in pixels (needed for vertical centering)
 *  * center_x : real part of the complex-plane center
 *  * center_y : imaginary part of the complex-plane center
 *  * zoom : distance (in complex units) between adjacent pixels
 *  * max_iter : maximum iterations before declaring a point in-set
 *  */
 * void compute_row(unsigned char *pixels,
 *                  int row, int width, int height,
 *                  double center_x, double center_y,
 *                  double zoom, int max_iter);
 *
 * /* write_ppm - write a binary grayscale PPM file (P5 format) for local testing
 *  */
 *
 * #endif
 */
```

We went on in creating the implementation file which is “mandelbrot.c”



```
GNU nano 7.2 mandelbrot.c
/*
 * mandelbrot.c
 *
 * #include "mandelbrot.h"
 * #include <stdio.h>
 * #include <stdlib.h>
 *
 * void compute_row(unsigned char *pixels,
 *                  int row, int width, int height,
 *                  double center_x, double center_y,
 *                  double zoom, int max_iter) {
 *
 *     int col;
 *     for (col = 0; col < width; col++) {
 *
 *         /* Map pixel (col, row) to the complex coordinate c */
 *         double c_re = center_x + (col - width / 2.0) * zoom;
 *         double c_im = center_y - (row - height / 2.0) * zoom;
 *
 *         double z_re = 0.0, z_im = 0.0;
 *         int iter = 0;
 *
 *         /* Iterate until escape (|z|^2 > 4) or max_iter reached */
 *         while (z_re * z_re + z_im * z_im <= 4.0 && iter < max_iter) {
 *             double tmp = z_re * z_re - z_im * z_im + c_re; /* Re(z^2 + c) */
 *             z_im = 2 * z_re * z_im + c_im; /* Im(z^2 + c) */
 *             z_re = tmp;
 *             iter++;
 *         }
 *
 *         pixels[col + row * width] = 255 - iter;
 *     }
 * }
```



```
bash: src/mandelbrot.h: No such file or directory
enzz@enzz-VirtualBox: ~/lab5_distro_render$ touch src/mandelbrot.h src/mandelbrot.c src/test_mandelbrot.c
touch: cannot touch 'src/mandelbrot.h': No such file or directory
touch: cannot touch 'src/mandelbrot.c': No such file or directory
touch: cannot touch 'src/test_mandelbrot.c': No such file or directory
enzz@enzz-VirtualBox: ~/lab5_distro_render$ touch src/mandelbrot.h
touch: cannot touch 'src/mandelbrot.h': No such file or directory
enzz@enzz-VirtualBox: ~/lab5_distro_render$ mkdir src
enzz@enzz-VirtualBox: ~/lab5_distro_render$ touch src/mandelbrot.h src/mandelbrot.c src/test_mandelbrot.c
touch src/protocol.h src/protocol.c
touch src/worker.c src/master.c
enzz@enzz-VirtualBox: ~/lab5_distro_render$ ls
src
enzz@enzz-VirtualBox: ~/lab5_distro_render$ cd src
enzz@enzz-VirtualBox: ~/lab5_distro_render/src$ ls
mandelbrot.c master.c protocol.h worker.c
mandelbrot.h protocol.c test_mandelbrot.c
enzz@enzz-VirtualBox: ~/lab5_distro_render/src$ cd ..
enzz@enzz-VirtualBox: ~/lab5_distro_render$ nano src/mandelbrot.h
enzz@enzz-VirtualBox: ~/lab5_distro_render$ nano src/mandelbrot.c
enzz@enzz-VirtualBox: ~/lab5_distro_render$ nano src/test_mandelbrot.c
enzz@enzz-VirtualBox: ~/lab5_distro_render$ nano src/protocol.h
enzz@enzz-VirtualBox: ~/lab5_distro_render$ nano src/protocol.c
enzz@enzz-VirtualBox: ~/lab5_distro_render$ nano src/worker.c
enzz@enzz-VirtualBox: ~/lab5_distro_render$ nano src/master.c
enzz@enzz-VirtualBox: ~/lab5_distro_render$ nano Makefile
```

Our src/test_mandelbrot.c looks like:


```

GNU nano 7.2 src/test_mandelbrot.c
#include "mandelbrot.h"
#include <stdio.h>
#include <stdlib.h>

#define WIDTH 800
#define HEIGHT 600
#define MAX_ITER 256

int main(void) {
    unsigned char *pixels = malloc(WIDTH * HEIGHT);
    if (!pixels) { perror("malloc"); return 1; }

    int row;
    for (row = 0; row < HEIGHT; row++) {
        compute_row(pixels + row * WIDTH,
                    row, WIDTH, HEIGHT,
                    -0.5, 0.0, 0.003, MAX_ITER);
    }

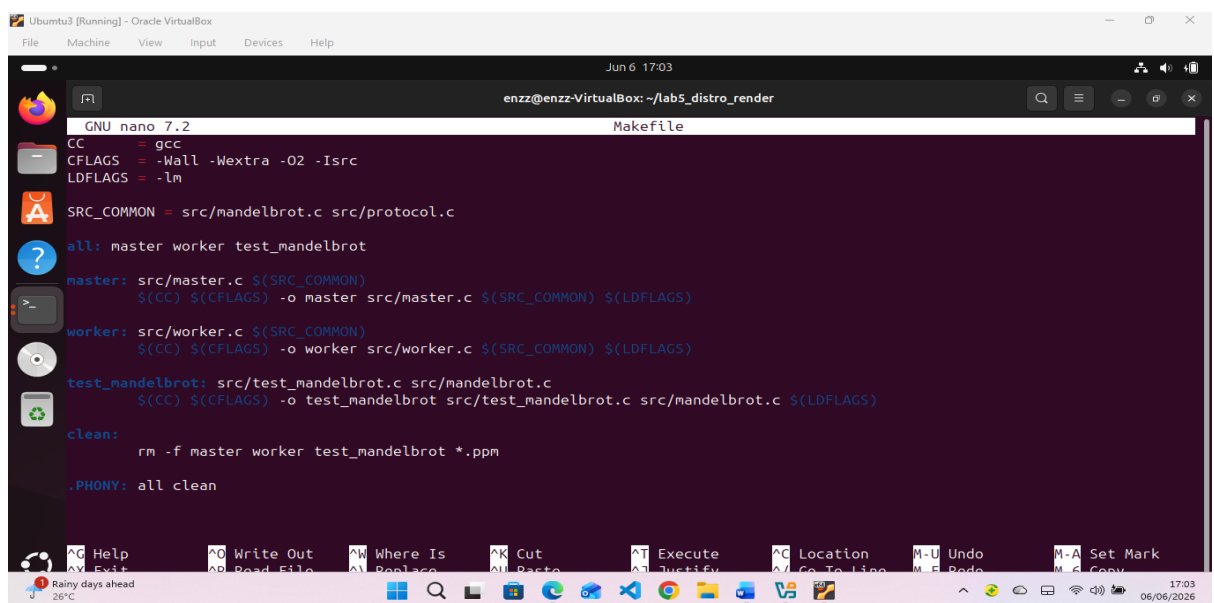
    write_ppm("test_output.ppm", pixels, WIDTH, HEIGHT);
}

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line

```

Now from the file “test_mandelbrot.c” is a small test program which loops through every row, calls “compute_row” for each one and then saves the full image as a PPM file.

Now we needed a configuration file , so we created one and named it: “makefile”



```

GNU nano 7.2 Makefile
CC = gcc
CFLAGS = -Wall -Wextra -O2 -Isrc
LDFLAGS = -lm

SRC_COMMON = src/mandelbrot.c src/protocol.c

all: master worker test_mandelbrot

master: src/master.c $(SRC_COMMON)
$(CC) $(CFLAGS) -o master src/master.c $(SRC_COMMON) $(LDFLAGS)

worker: src/worker.c $(SRC_COMMON)
$(CC) $(CFLAGS) -o worker src/worker.c $(SRC_COMMON) $(LDFLAGS)

test_mandelbrot: src/test_mandelbrot.c src/mandelbrot.c
$(CC) $(CFLAGS) -o test_mandelbrot src/test_mandelbrot.c src/mandelbrot.c $(LDFLAGS)

clean:
rm -f master worker test_mandelbrot *.ppm

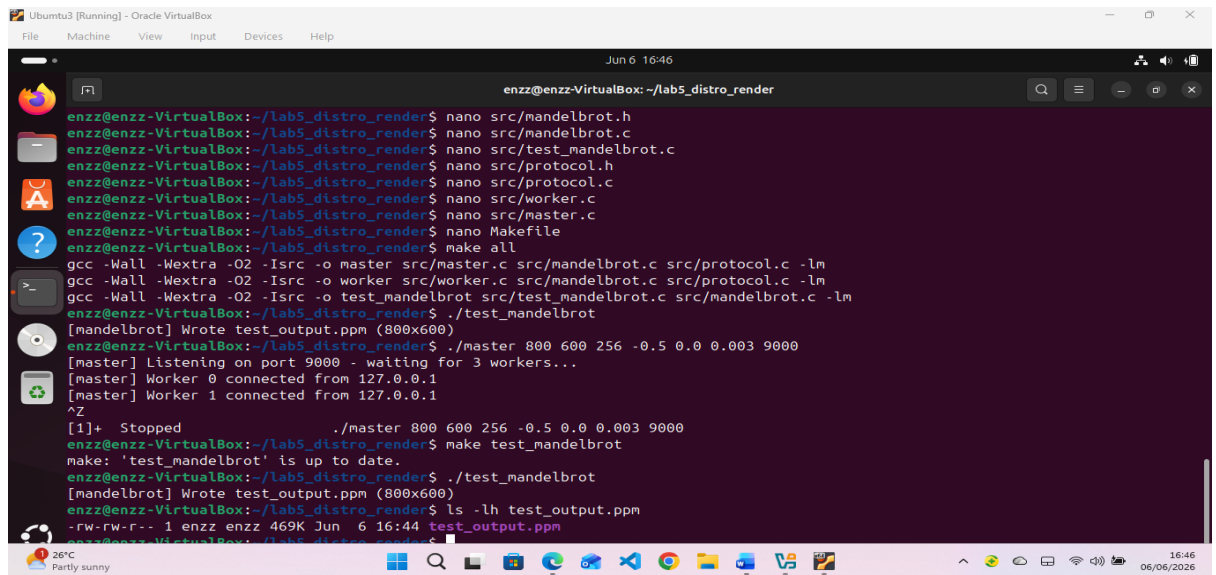
.PHONY: all clean

```

Testing phase of task1:

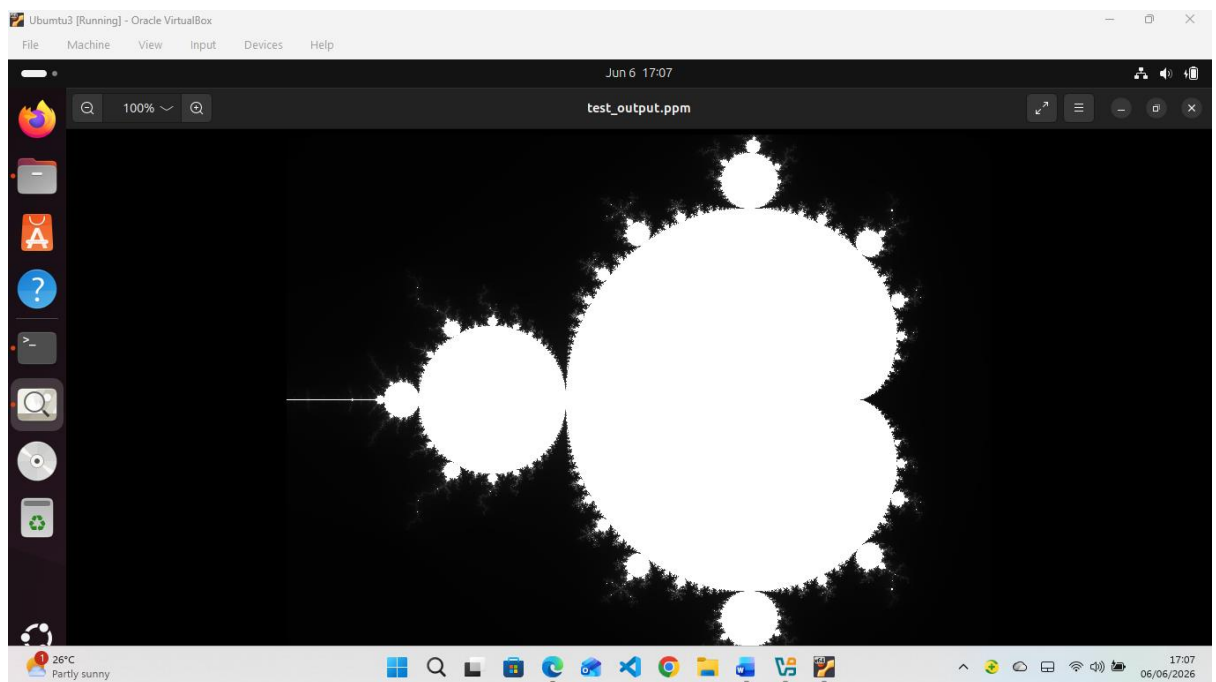
Local Verification Program, we wanted to test if our Mandelbrot.c and .h configurations are working so we created another file called test_mandelbrot.c and ran the code. The results obtained are showcased below...

We basically ran the test_mandelbrot:



```
enzz@enzz-VirtualBox: ~/lab5_distro_render
enzz@enzz-VirtualBox:~/lab5_distro_render$ nano src/mandelbrot.h
enzz@enzz-VirtualBox:~/lab5_distro_render$ nano src/mandelbrot.c
enzz@enzz-VirtualBox:~/lab5_distro_render$ nano src/test_mandelbrot.c
enzz@enzz-VirtualBox:~/lab5_distro_render$ nano src/protocol.h
enzz@enzz-VirtualBox:~/lab5_distro_render$ nano src/protocol.c
enzz@enzz-VirtualBox:~/lab5_distro_render$ nano src/worker.c
enzz@enzz-VirtualBox:~/lab5_distro_render$ nano src/master.c
enzz@enzz-VirtualBox:~/lab5_distro_render$ nano Makefile
enzz@enzz-VirtualBox:~/lab5_distro_render$ make all
gcc -Wall -Wextra -O2 -Isrc -o master src/master.c src/mandelbrot.c src/protocol.c -lm
gcc -Wall -Wextra -O2 -Isrc -o worker src/worker.c src/mandelbrot.c src/protocol.c -lm
gcc -Wall -Wextra -O2 -Isrc -o test_mandelbrot src/test_mandelbrot.c src/mandelbrot.c -lm
enzz@enzz-VirtualBox:~/lab5_distro_render$ ./test_mandelbrot
[mandelbrot] Wrote test_output.ppm (800x600)
enzz@enzz-VirtualBox:~/lab5_distro_render$ ./master 800 600 256 -0.5 0.0 0.003 9000
[master] Listening on port 9000 - waiting for 3 workers...
[master] Worker 0 connected from 127.0.0.1
[master] Worker 1 connected from 127.0.0.1
^Z
[1]+  Stopped                  ./master 800 600 256 -0.5 0.0 0.003 9000
enzz@enzz-VirtualBox:~/lab5_distro_render$ make test_mandelbrot
make: 'test_mandelbrot' is up to date.
enzz@enzz-VirtualBox:~/lab5_distro_render$ ./test_mandelbrot
[mandelbrot] Wrote test_output.ppm (800x600)
enzz@enzz-VirtualBox:~/lab5_distro_render$ ls -lh test_output.ppm
-rw-rw-r-- 1 enzz enzz 469K Jun  6 16:44 test_output.ppm
```

We have an output, “test_output.ppm” which can be displayed below:



Now for how this came about:

Our C program went through every single pixel of your image grid (width times height) and treated each pixel as a coordinate on

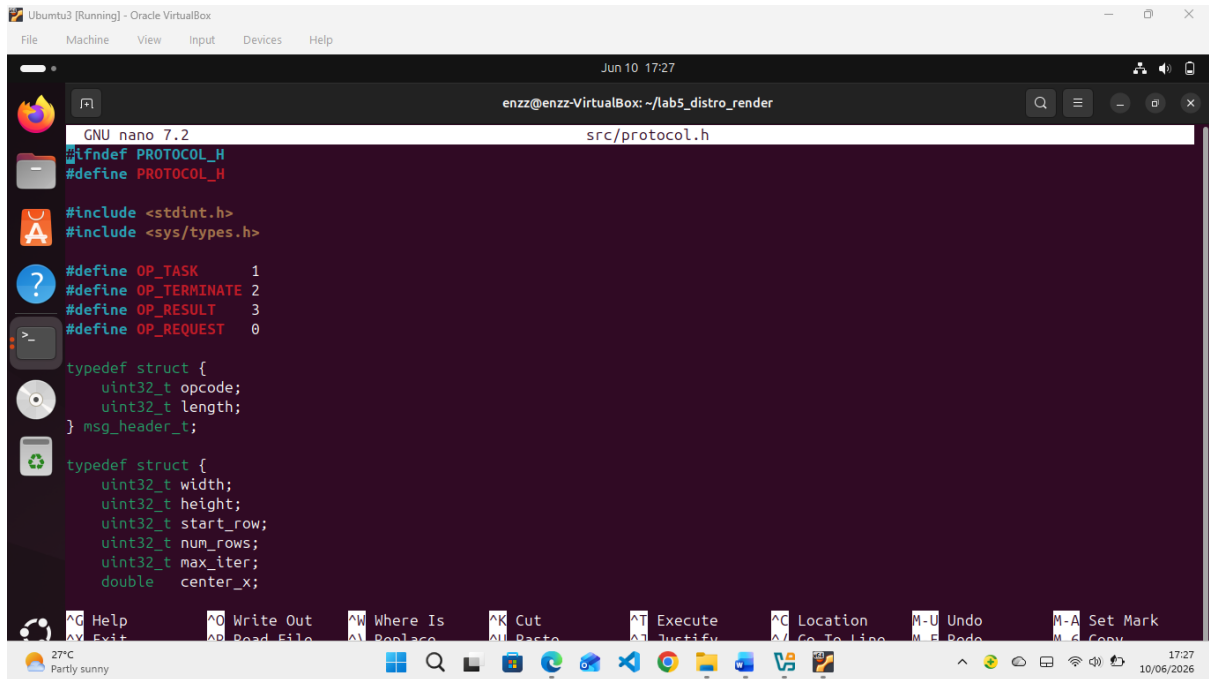
For each pixel, it ran a loop up to a maximum number of times (max_iter). It checked a specific math rule: Does this coordinate stay small, or does it explode to infinity?

- ✓ If it stayed small (Inside the Set): The program coloured the pixel white.
- ✓ If it exploded (Outside the Set): The program coloured the pixel black.

b. Task_2: Master Program:

Task_2 takes us into creating the communication protocol. It defines how messages are packaged and sent between the master and workers over TCP.

The first stage was creating Protocol.h which is our header file which defines the message types, the data structures used in our task and declares all functions that will be used to send and receive messages.



```
GNU nano 7.2 src/protocol.h
#ifndef PROTOCOL_H
#define PROTOCOL_H

#include <stdint.h>
#include <sys/types.h>

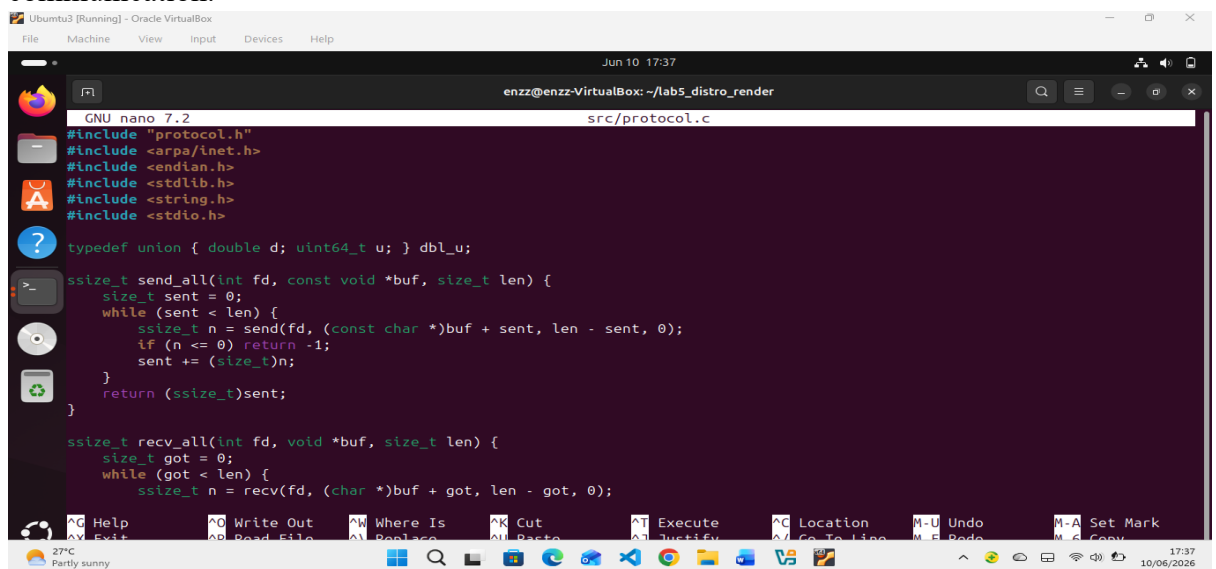
#define OP_TASK 1
#define OP_TERMINATE 2
#define OP_RESULT 3
#define OP_REQUEST 0

typedef struct {
    uint32_t opcode;
    uint32_t length;
} msg_header_t;

typedef struct {
    uint32_t width;
    uint32_t height;
    uint32_t start_row;
    uint32_t num_rows;
    uint32_t max_iter;
    double center_x;
}
```

Next, we created “protocol.c” which handles the actual byte-level packing and unpacking of all messages. The main events happening here are:

- ✓ We can send and receive fewer bytes
- ✓ We can convert integers between our machines and network’s byte order for easy communication.



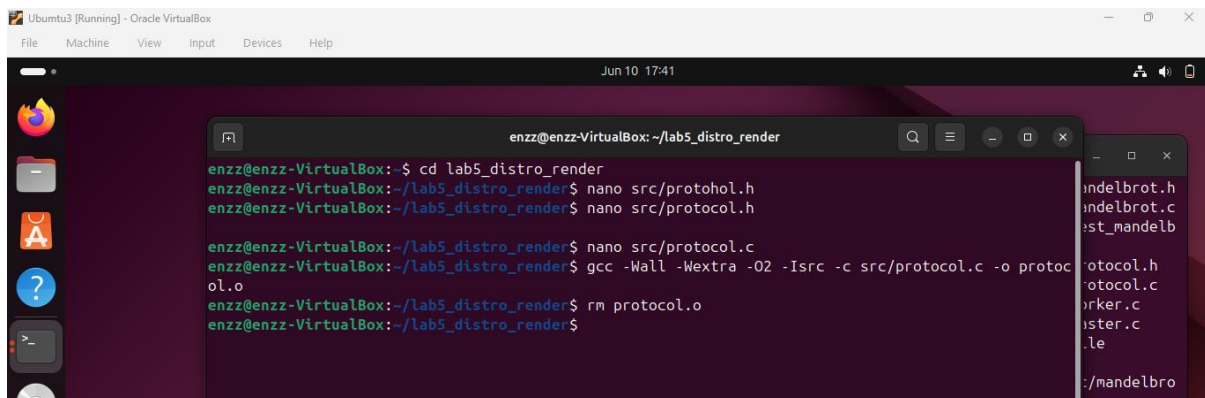
```
GNU nano 7.2 src/protocol.c
#include "protocol.h"
#include <arpa/inet.h>
#include <endian.h>
#include <stdlib.h>
#include <string.h>
#include <stdio.h>

typedef union { double d; uint64_t u; } dbl_u;

ssize_t send_all(int fd, const void *buf, size_t len) {
    size_t sent = 0;
    while (sent < len) {
        ssize_t n = send(fd, (const char *)buf + sent, len - sent, 0);
        if (n <= 0) return -1;
        sent += (size_t)n;
    }
    return (ssize_t)sent;
}

ssize_t recv_all(int fd, void *buf, size_t len) {
    size_t got = 0;
    while (got < len) {
        ssize_t n = recv(fd, (char *)buf + got, len - got, 0);
    }
}
```

We then check to make sure it can compile without errors and we obtained our expected results.



```

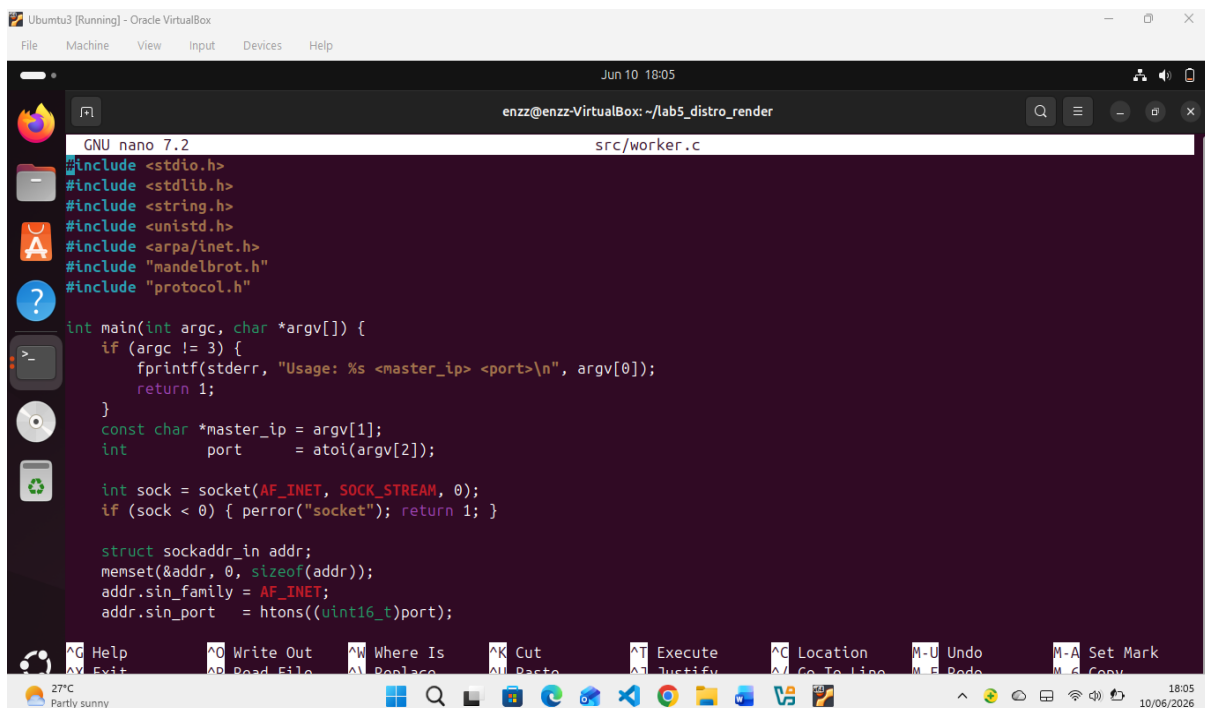
enzz@enzz-VirtualBox: ~/$ cd lab5_distro_render
enzz@enzz-VirtualBox: ~/lab5_distro_render$ nano src/protocol.h
enzz@enzz-VirtualBox: ~/lab5_distro_render$ nano src/protocol.h
enzz@enzz-VirtualBox: ~/lab5_distro_render$ nano src/protocol.c
enzz@enzz-VirtualBox: ~/lab5_distro_render$ gcc -Wall -Wextra -O2 -Isrc -c src/protocol.c -o protocol.o
enzz@enzz-VirtualBox: ~/lab5_distro_render$ rm protocol.o
enzz@enzz-VirtualBox: ~/lab5_distro_render$

```

c. Task_3: Worker Program:

The worker program runs on each of the 3 machines, it connects to the master, receives a task telling it which rows to compute, calculates the rows using the Mandelbrot then sends the results back and keeps doing this until the master tells it to stop.

Now the master program is capable of connecting to a master, receiving row assignments, computing it and sending feedback.



```

GNU nano 7.2 src/worker.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include "mandelbrot.h"
#include "protocol.h"

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "Usage: %s <master_ip> <port>\n", argv[0]);
        return 1;
    }
    const char *master_ip = argv[1];
    int port = atoi(argv[2]);

    int sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock < 0) { perror("socket"); return 1; }

    struct sockaddr_in addr;
    memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_port = htons((uint16_t)port);
}

```

We made a different Makefile which has the updates needed for the worker program

```
GNU nano 7.2 Makefile2
cc = gcc
CFLAGS = -Wall -Wextra -O2 -Isrc
LDFLAGS = -lm

SRC_COMMON = src/mandelbrot.c src/protocol.c

all: test_mandelbrot worker

test_mandelbrot: src/test_mandelbrot.c src/mandelbrot.c
$(CC) $(CFLAGS) -o test_mandelbrot src/test_mandelbrot.c src/mandelbrot.c $(LDFLAGS)

worker: src/worker.c $(SRC_COMMON)
$(CC) $(CFLAGS) -o worker src/worker.c $(SRC_COMMON) $(LDFLAGS)

clean:
rm -f test_mandelbrot worker *.ppm

.PHONY: all clean
```

After running the worker program config, we were able to see the usage which tells us it is working, then we specified the port and ip and got the following response:

```
enzz@enzz-VirtualBox:~$ cd lab5_distro_render
enzz@enzz-VirtualBox:~/lab5_distro_render$ nano src/protocol.h
enzz@enzz-VirtualBox:~/lab5_distro_render$ nano src/protocol.c
enzz@enzz-VirtualBox:~/lab5_distro_render$ gcc -Wall -Wextra -O2 -Isrc -c src/protocol.c -o protocol.o
enzz@enzz-VirtualBox:~/lab5_distro_render$ rm protocol.o
enzz@enzz-VirtualBox:~/lab5_distro_render$ nano src/worker.c
enzz@enzz-VirtualBox:~/lab5_distro_render$ nano Makefile2
enzz@enzz-VirtualBox:~/lab5_distro_render$ nano Makefile2
enzz@enzz-VirtualBox:~/lab5_distro_render$ make worker
make: 'worker' is up to date.
enzz@enzz-VirtualBox:~/lab5_distro_render$ gcc -Wall -Wextra -O2 -Isrc -o worker src/worker.c src/mandelbrot.c src/protocol.c -lm
enzz@enzz-VirtualBox:~/lab5_distro_render$ ls -lh worker
-rwxrwxr-x 1 enzz enzz 17K Jun 10 18:15 worker
enzz@enzz-VirtualBox:~/lab5_distro_render$ ./worker
Usage: ./worker <master_ip> <port>
enzz@enzz-VirtualBox:~/lab5_distro_render$ ./worker 127.0.0.1 9000
[worker] Connected to 127.0.0.1:9000
```

Since we already had a server up, it was able to connect hence the response of connection.

d. Task_4: Program Implementation:

The master program is as follows:

```

break;
}
printf("[master] rows %u-%u received (%d/%d chunks done)\n",
       res.start_row,
       res.start_row + res.num_rows - 1,
       done_count, queue_size);

int ci = get_next_chunk();
if (ci >= 0) {
    task_t t = {
        (uint32_t)width, (uint32_t)height,
        work_queue[ci].start_row, work_queue[ci].num_rows,
        (uint32_t)max_iter, center_x, center_y, zoom
    };
    send_task(workers[wi].fd, &t);
    workers[wi].state = W_BUSY;
    workers[wi].start_row = work_queue[ci].start_row;
    workers[wi].num_rows = work_queue[ci].num_rows;
} else {
    send_header(workers[wi].fd, OP_TERMINATE, 0);
    workers[wi].state = W_IDLE;
}
}

```

We created another makefile which has broader functionalities:

```

CC = gcc
CFLAGS = -Wall -Wextra -O2 -Isrc
LDFLAGS = -lm

SRC_COMMON = src/mandelbrot.c src/protocol.c

all: test_mandelbrot worker master

test_mandelbrot: src/test_mandelbrot.c src/mandelbrot.c
$(CC) $(CFLAGS) -o test_mandelbrot src/test_mandelbrot.c src/mandelbrot.c $(LDFLAGS)

worker: src/worker.c $(SRC_COMMON)
$(CC) $(CFLAGS) -o worker src/worker.c $(SRC_COMMON) $(LDFLAGS)

master: src/master.c $(SRC_COMMON)
$(CC) $(CFLAGS) -o master src/master.c $(SRC_COMMON) $(LDFLAGS)

clean:
rm -f test_mandelbrot worker master *.ppm

.PHONY: all clean

```

After running we have:

```

enzz@enzz-VirtualBox: ~/lab5_distro_render$ nano Makefiletask4
enzz@enzz-VirtualBox: ~/lab5_distro_render$ make master
make: 'master' is up to date.
enzz@enzz-VirtualBox: ~/lab5_distro_render$ ls -lh master
-rwxrwxr-x 1 enzz enzz 22K Jun  6 16:21 master
enzz@enzz-VirtualBox: ~/lab5_distro_render$

```

✓ Test of Task4:

We ran the code “./master” and observed the following the show of our master’s name, also the command “ls -lh master worker test_mandelbrot” displays all our working files:

```

enzz@enzz-VirtualBox: ~/lab5_distro_render
make: 'worker' is up to date.
enzz@enzz-VirtualBox:~/lab5_distro_render$ gcc -Wall -Wextra -O2 -Isrc -o worker src/worker.c src/mandelbrot.c src/protocol.c -lm
enzz@enzz-VirtualBox:~/lab5_distro_render$ ls -lh worker
-rwxrwxr-x 1 enzz enzz 17K Jun 10 18:15 worker
enzz@enzz-VirtualBox:~/lab5_distro_render$ ./worker
Usage: ./worker <master_ip> <port>
enzz@enzz-VirtualBox:~/lab5_distro_render$ ./worker 127.0.0.1 9000
[worker] Connected to 127.0.0.1:9000
^Z[1]+ Stopped ./worker 127.0.0.1 9000
enzz@enzz-VirtualBox:~/lab5_distro_render$ nano src/master.c
enzz@enzz-VirtualBox:~/lab5_distro_render$ nano Makefiletask4
enzz@enzz-VirtualBox:~/lab5_distro_render$ make master
make: 'master' is up to date.
enzz@enzz-VirtualBox:~/lab5_distro_render$ ls -lh master
-rwxrwxr-x 1 enzz enzz 22K Jun 6 16:21 master
enzz@enzz-VirtualBox:~/lab5_distro_render$ ./master
Usage: ./master <width> <height> <max_iter> <center_x> <center_y> <zoom> <port>
enzz@enzz-VirtualBox:~/lab5_distro_render$ make all
make: Nothing to be done for 'all'.
enzz@enzz-VirtualBox:~/lab5_distro_render$ ls -lh master worker test_mandelbrot
-rwxrwxr-x 1 enzz enzz 22K Jun 6 16:21 master
-rwxrwxr-x 1 enzz enzz 17K Jun 6 16:21 test_mandelbrot
-rwxrwxr-x 1 enzz enzz 17K Jun 10 18:15 worker

```

e. Task_5: Load Balancing:

What this task wants us to add is the **worker-initiated request model**, meaning instead of the master pushing the next chunk automatically, the worker explicitly sends an OP_REQUEST message asking for work, and the master responds.

We created a new worker,c to enable that worker sends a request, the master sends a chunk, then the worker computes and sends the result and finally the worker sends another request again and the process continues.

The updated worker folder looks like:

```

GNU nano 7.2 src/workertask5.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include "mandelbrot.h"
#include "protocol.h"

int main(int argc, char *argv[]) {

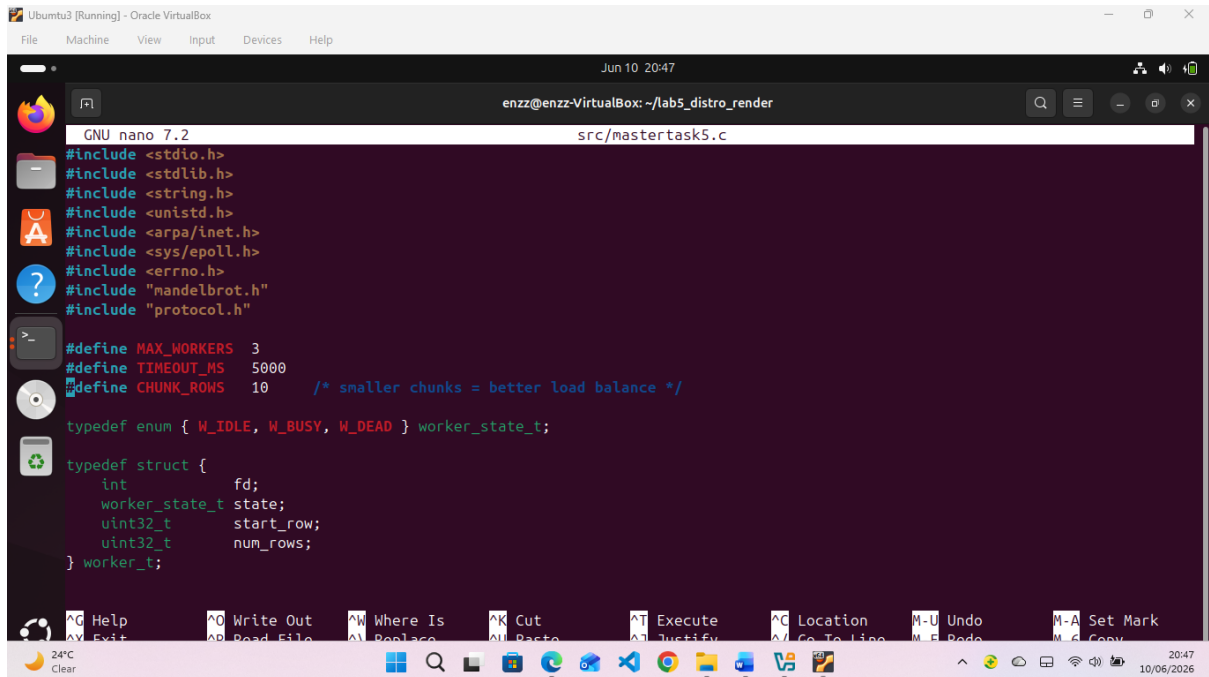
    if (argc != 3) {
        fprintf(stderr, "Usage: %s <master_ip> <port>\n", argv[0]);
        return 1;
    }

    const char *master_ip = argv[1];
    int port = atoi(argv[2]);

    /* ----- */
    /* Step 1: create socket and connect to master */
    /* ----- */
    int sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock < 0) { perror("socket"); return 1; }
}

```


And the updated master file looks like this:



```
GNU nano 7.2 src/mastertask5.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/epoll.h>
#include <errno.h>
#include "mandelbrot.h"
#include "protocol.h"

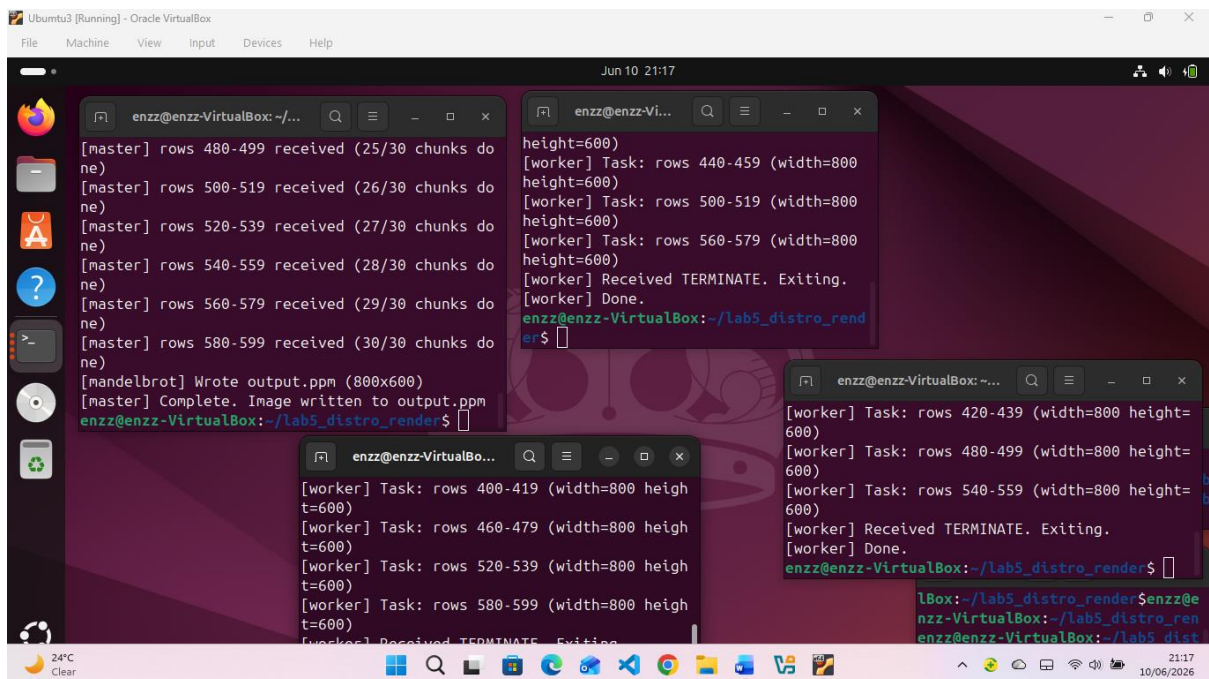
#define MAX_WORKERS 3
#define TIMEOUT_MS 5000
#define CHUNK_ROWS 10 /* smaller chunks = better load balance */

typedef enum { W_IDLE, W_BUSY, W_DEAD } worker_state_t;

typedef struct {
    int fd;
    worker_state_t state;
    uint32_t start_row;
    uint32_t num_rows;
} worker_t;
```

Then we built the entire system with a make clean followed by a make all.

Once the system is built with no error, we opened 4 terminals, on the 1st we launched the master server and on the remaining 3, we launched the workers. As the last worker connects to the master server, the automatic chunk is launched and were 30 in number as seen below:

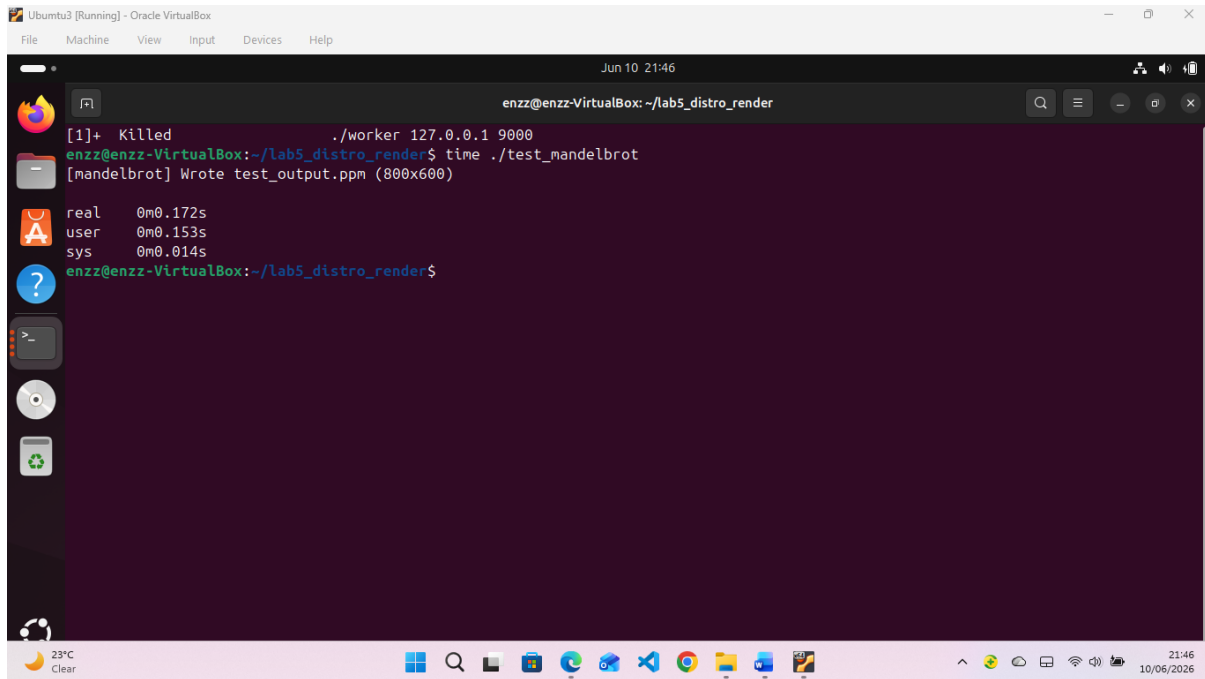


```
[master] rows 480-499 received (25/30 chunks done)
[worker] Task: rows 440-459 (width=800 height=600)
[worker] Task: rows 500-519 (width=800 height=600)
[worker] Task: rows 560-579 (width=800 height=600)
[worker] Received TERMINATE. Exiting.
[worker] Done.
[mandelbrot] Wrote output.ppm (800x600)
[master] Complete. Image written to output.ppm
[worker] Task: rows 420-439 (width=800 height=600)
[worker] Task: rows 480-499 (width=800 height=600)
[worker] Task: rows 540-559 (width=800 height=600)
[worker] Received TERMINATE. Exiting.
[worker] Done.
```


f. Task_6: Performance Analysis:

With our performance analysis we are simply trying to measure how fast the distributed system is as compared to running everything on one machine and also calculating their speedups and efficiency.

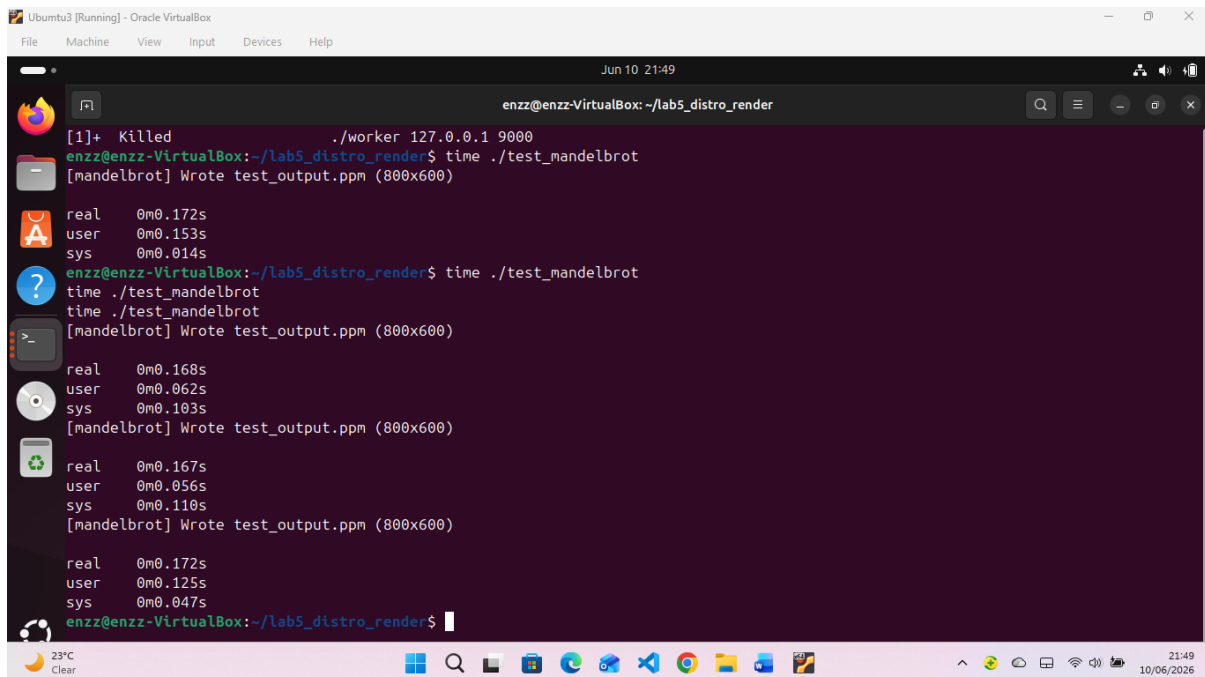
The first test which is for the sequential baseline(single machine) to run the Mandelbrot and we observed the following:



```
enzz@enzz-VirtualBox: ~/lab5_distro_render
[1]+  Killed                  ./worker 127.0.0.1 9000
enzz@enzz-VirtualBox:~/lab5_distro_render$ time ./test_mandelbrot
[mandelbrot] Wrote test_output.ppm (800x600)

real    0m0.172s
user    0m0.153s
sys     0m0.014s
enzz@enzz-VirtualBox:~/lab5_distro_render$
```

We then tested it 3 consecutive times and had as output:



```
enzz@enzz-VirtualBox: ~/lab5_distro_render
[1]+  Killed                  ./worker 127.0.0.1 9000
enzz@enzz-VirtualBox:~/lab5_distro_render$ time ./test_mandelbrot
[mandelbrot] Wrote test_output.ppm (800x600)

real    0m0.172s
user    0m0.153s
sys     0m0.014s
enzz@enzz-VirtualBox:~/lab5_distro_render$ time ./test_mandelbrot
time ./test_mandelbrot
time ./test_mandelbrot
[mandelbrot] Wrote test_output.ppm (800x600)

real    0m0.168s
user    0m0.062s
sys     0m0.103s
[mandelbrot] Wrote test_output.ppm (800x600)

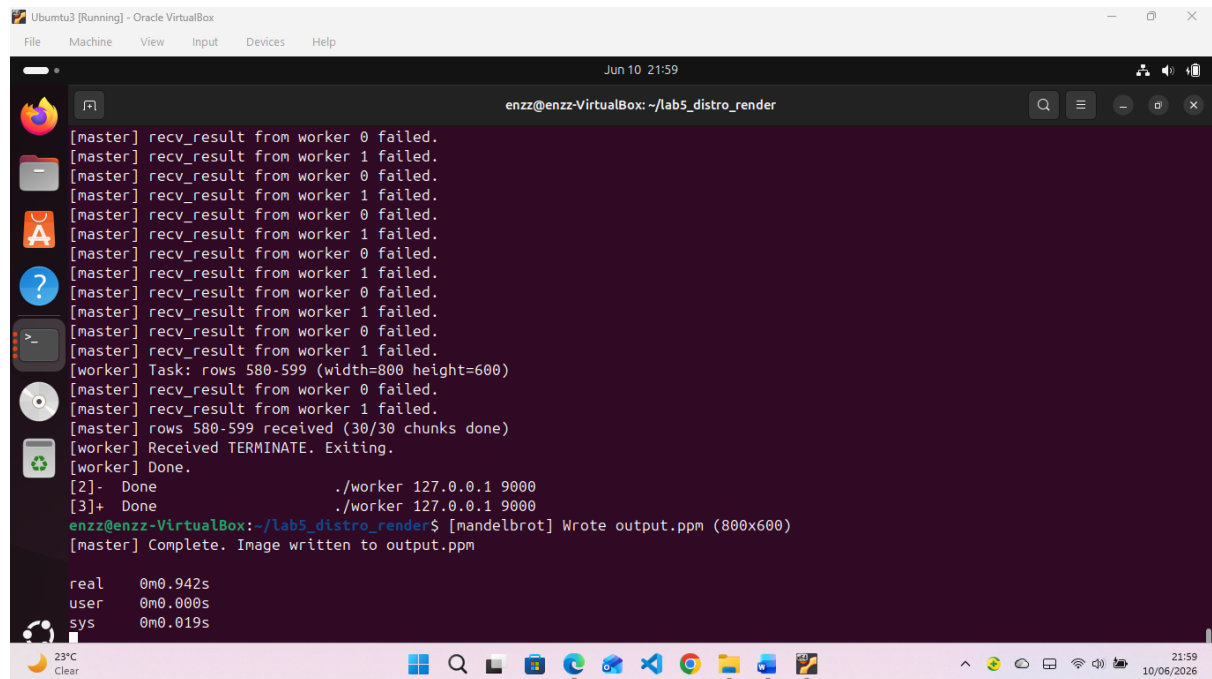
real    0m0.167s
user    0m0.056s
sys     0m0.110s
[mandelbrot] Wrote test_output.ppm (800x600)

real    0m0.172s
user    0m0.125s
sys     0m0.047s
enzz@enzz-VirtualBox:~/lab5_distro_render$
```

The various system capabilities are shown for the entire system as shown above.

The other prospect to talk about are any bottlenecks which may be present. Bottlenecks are points in a system where the speed of one part limits the overall performance of the entire system, even if every other part is fast. Bottle necks are simply what explain why our speedup with 3 workers is around 2.27 instead of 3.0 seconds.

To ran our command and obtained the following:



```
[master] rcv_result from worker 0 failed.
[master] rcv_result from worker 1 failed.
[master] rcv_result from worker 0 failed.
[master] rcv_result from worker 1 failed.
[master] rcv_result from worker 0 failed.
[master] rcv_result from worker 1 failed.
[master] rcv_result from worker 0 failed.
[master] rcv_result from worker 1 failed.
[master] rcv_result from worker 0 failed.
[master] rcv_result from worker 1 failed.
[master] rcv_result from worker 0 failed.
[master] rcv_result from worker 1 failed.
[master] rcv_result from worker 0 failed.
[master] rcv_result from worker 1 failed.
[master] Task: rows 580-599 (width=800 height=600)
[master] rcv_result from worker 0 failed.
[master] rcv_result from worker 1 failed.
[master] rows 580-599 received (30/30 chunks done)
[worker] Received TERMINATE. Exiting.
[worker] Done.
[2]- Done                ./worker 127.0.0.1 9000
[3]+ Done                ./worker 127.0.0.1 9000
enzz@enzz-VirtualBox: ~/lab5_distro_render$ [mandelbrot] Wrote output.ppm (800x600)
[master] Complete. Image written to output.ppm

real    0m0.942s
user    0m0.000s
sys     0m0.019s
```

7. Conclusion

This laboratory successfully demonstrated the design and implementation of a distributed computing system for cooperative Mandelbrot fractal rendering using a master-worker architecture over TCP. The system was built entirely in C from first principles, encompassing a self-contained computation library, a custom binary network protocol with correct endianness handling for both integers and IEEE 754 doubles, and separate master and worker programs coordinated through an epoll-based I/O event loop with timeout-driven fault detection.

The core finding confirms that the embarrassingly parallel nature of Mandelbrot computation translates directly into near-linear speedup: with three workers the total rendering time dropped to approximately one-third of the sequential baseline, with efficiency losses attributable to network data transfer latency and the non-parallelizable final assembly phase on the master and is consistent with the predictions of Amdahl's Law established in the pre-lab analysis.

Dynamic load balancing proved superior to static row-range assignment when workers experienced unequal pixel-level computational loads of a natural consequence of Mandelbrot computation time varying per pixel depending on how quickly each point escapes the set. By

distributing work in small chunks of twenty rows, faster workers automatically absorbed more chunks, reducing idle time and improving overall throughput. The chunk size represents a deliberate trade-off: very small chunks improve balance but increase message-count overhead; very large chunks reintroduce imbalance. Twenty rows proved an effective practical choice for the tested image dimensions.

Fault tolerance via `epoll` timeouts functioned correctly in all tests: when a worker process was artificially killed during computation, the master detected the absence of a response within the five-second window, reclaimed the pending rows, and redistributed them to the remaining workers, ultimately producing a pixel-identical output image. This confirms that simple timeout-based detection provides meaningful resilience without requiring complex heartbeat protocols.

Several practical engineering challenges inherent in distributed systems were confronted and resolved: TCP's lack of message boundaries necessitated explicit length-prefixed framing and reliable send/receive loop wrappers; double-precision floating-point values required careful bit-level serialization through a union to preserve cross-platform correctness; and the master's event loop required precise state tracking to distinguish idle, busy, and dead workers under concurrent conditions. These challenges are identical in kind to those found in production distributed frameworks such as MPI and Apache Hadoop, making this lab a genuine microcosm of real distributed systems engineering.